

第十二讲：基于CNN的图像识别



授课人：王国泰 wgt_cv@163.com

电子科技大学机械与电气工程学院

2026-4-9

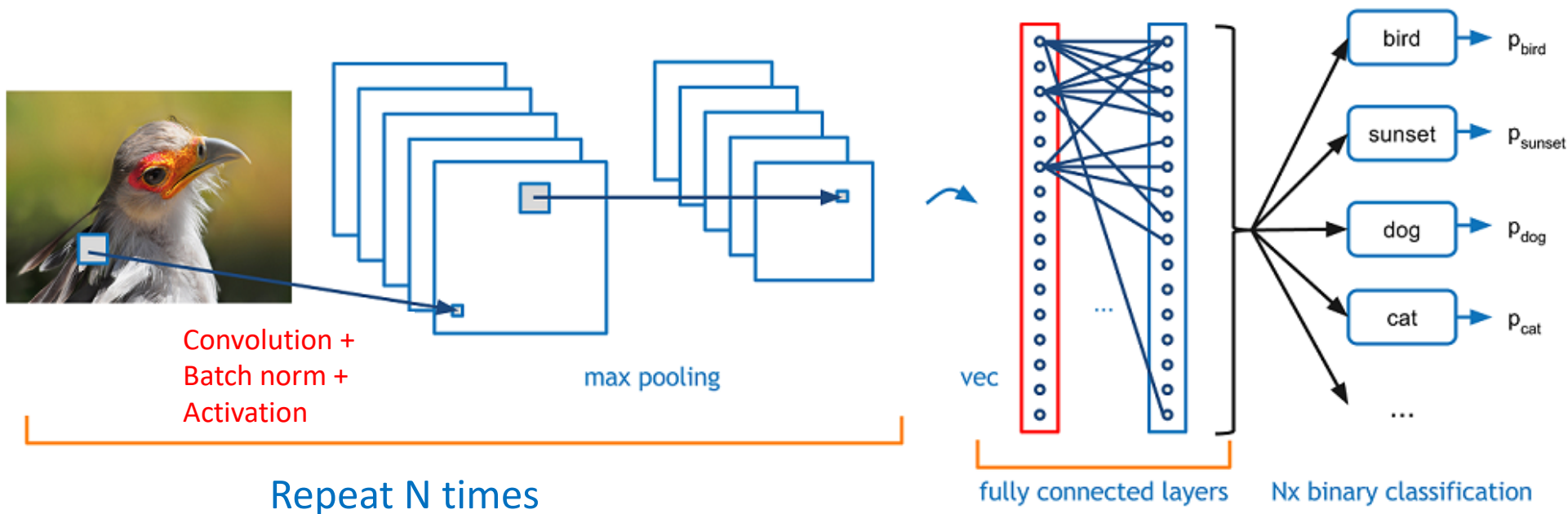


知识回顾：深度神经网络

- Convolutional Neural Network

卷积神经网络

- Mostly used for computer vision
- Can also be used for speech recognition



知识回顾：卷积层



Parameters of convolution

- Input and output channel
- Kernel size
- Padding
- Stride
- Dilation rate
- Group number

Pytorch
code

CONV2D

```
CLASS torch.nn.Conv2d(in_channels: int, out_channels: int,  
kernel_size: Union[T, Tuple[T, T]], stride: Union[T, Tuple[T,  
T]] = 1, padding: Union[T, Tuple[T, T]] = 0, dilation:  
Union[T, Tuple[T, T]] = 1, groups: int = 1, bias: bool =  
True, padding_mode: str = 'zeros')
```

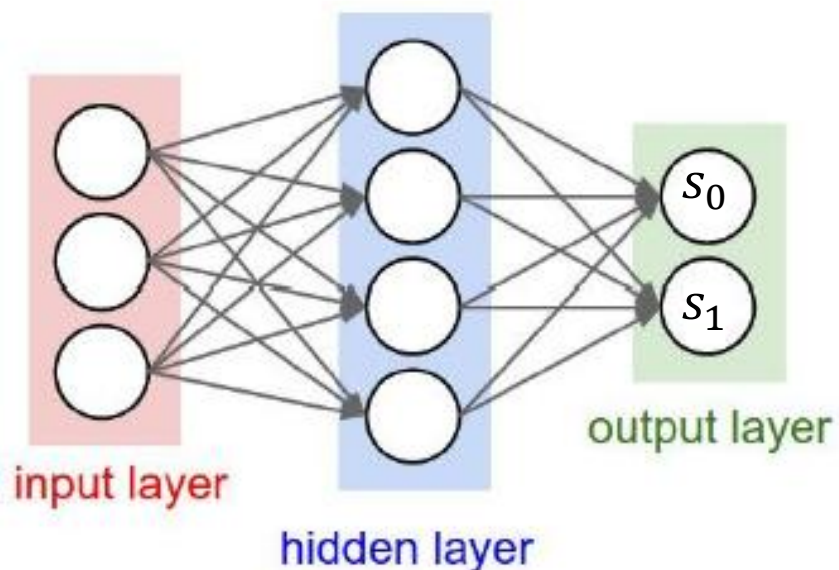
[SOURCE] [↗](#)

Tensorflow
code

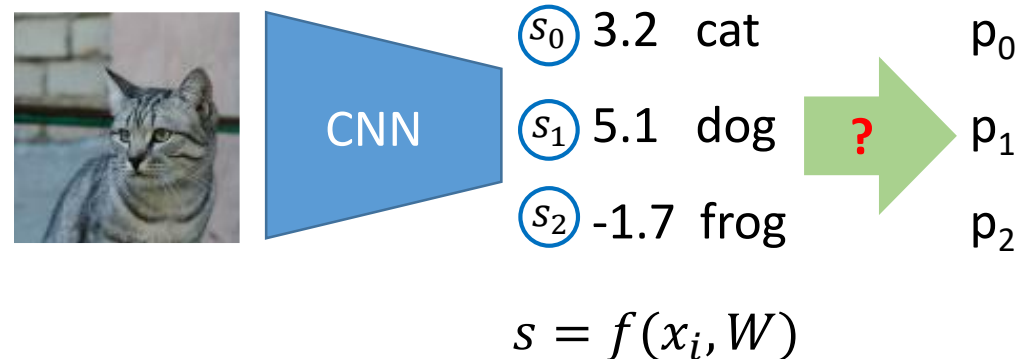
```
tf.keras.layers.Conv2D(  
    filters, kernel_size, strides=(1, 1), padding='valid',  
    data_format=None, dilation_rate=(1, 1), groups=1, activation=None,  
    use_bias=True, kernel_initializer='glorot_uniform',  
    bias_initializer='zeros', kernel_regularizer=None,  
    bias_regularizer=None, activity_regularizer=None, kernel_constraint=None,  
    bias_constraint=None, **kwargs  
)
```

Softmax与损失函数

- 神经网络中每个输出节点得到一个与该类别相关的分数
- 如何将这些分数转换为**概率值**?



提示：每个值需要为正数，并且之和为1

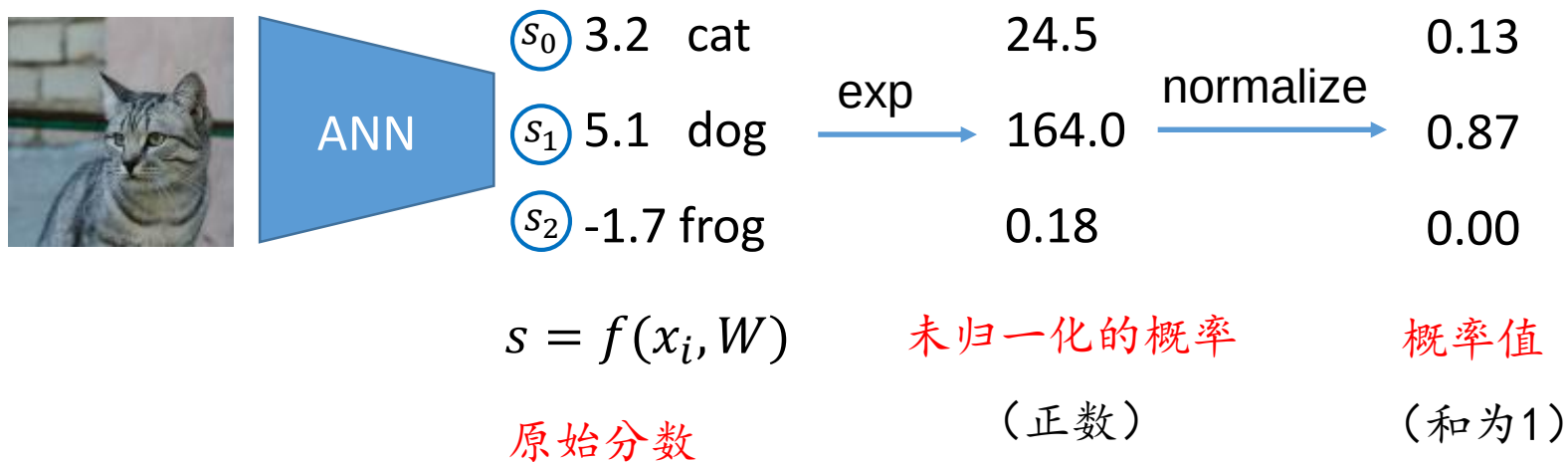


Softmax分类器

- 将输出节点的分数转换为**概率值**

Softmax

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$



知识回顾: Softmax与损失函数

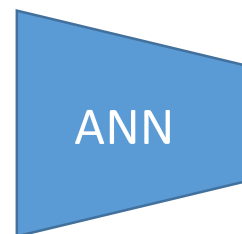
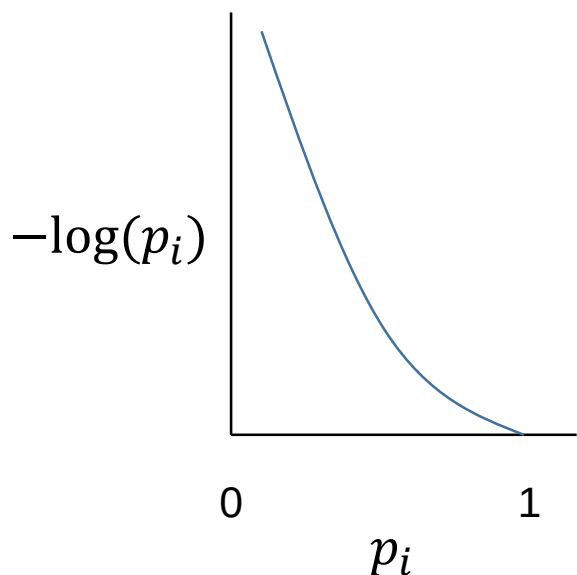


Softmax分类器

- 将输出节点的分数转换为**概率值**

$$L_i = -\log P(Y = y_i | X = x_i)$$

$$L_i = -\log(0.13) = -\sum_j y_j \log p_j = 2.04$$



s_0 3.2 cat
 s_1 5.1 dog
 s_2 -1.7 frog

softmax

0.13
0.87
0.00

$y_0 = 1$
 $y_1 = 0$
 $y_2 = 0$

$s = f(x_i, W)$

原始分数

概率值
(和为1)

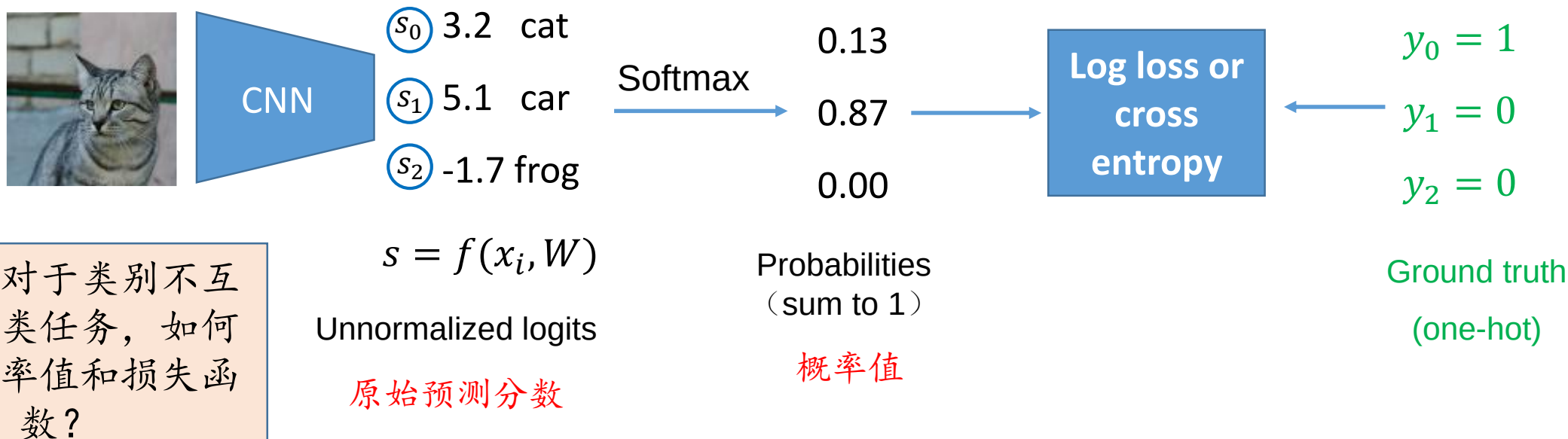
Ground truth

Softmax与损失函数



• Softmax 与Log损失联合使用?

- Softmax 通常用于分类网络的最后一层，得到各类别的概率（类别互斥）
- 预测的概率值输入交叉熵损失函数（log 损失）



Content



- 典型的图像分类网络
 - AlexNet, VGG
 - GoogleNet, ResNet
 - MobileNet, ShuffleNet
- 对CNN特征的解释

Typical network structures



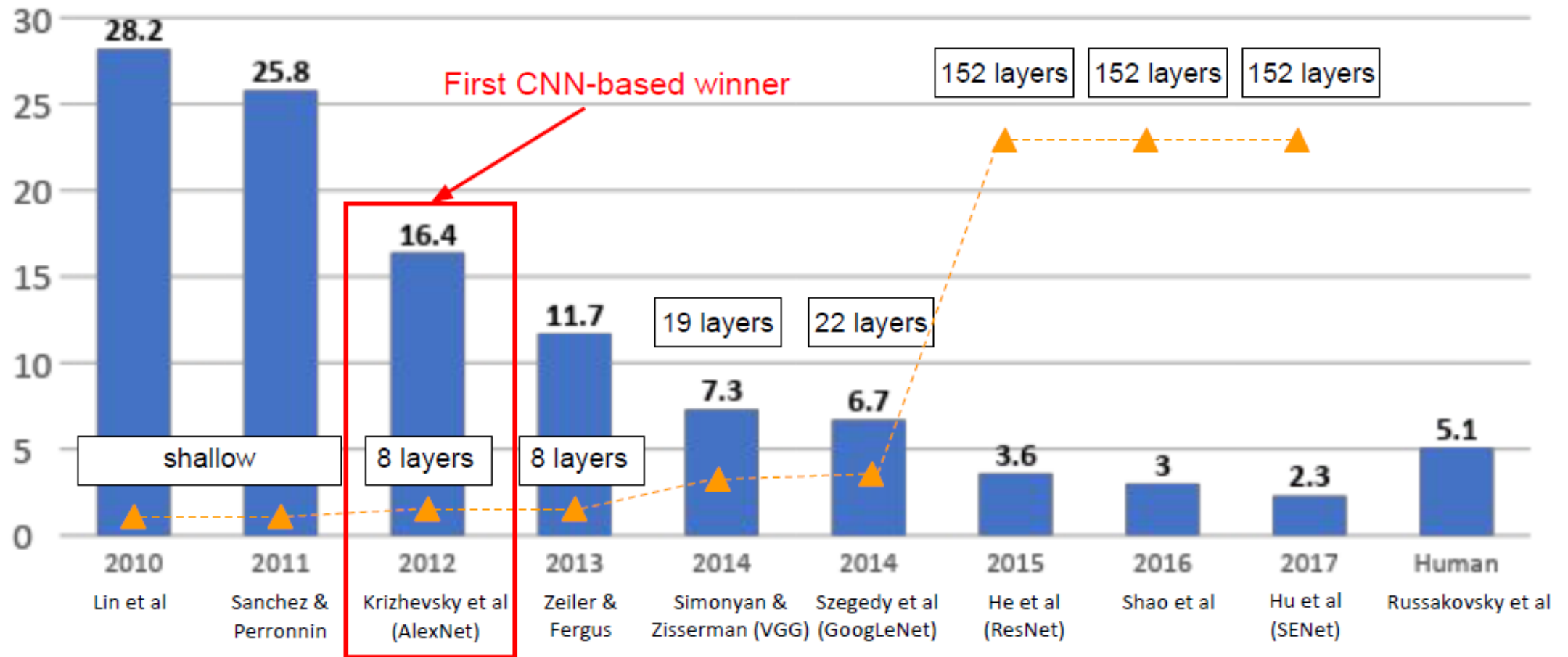
Widely used networks

- AlexNet
- VGG
- GoogleNet
- ResNet

More advanced networks

- SENet
- ResNeXT
- NeseNet
- MobileNets
- NasNet
- EfficientNet

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



Typical network structures

- AlexNet (8 layers)

Architecture:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

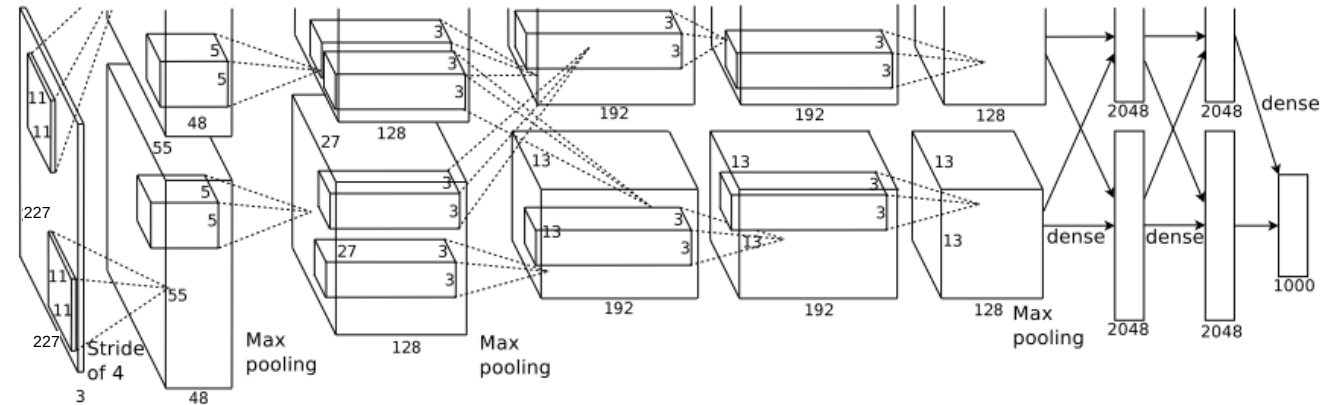
Max POOL3

FC6

FC7

FC8

Input:
227x227x3
images



First layer (CONV1): 96 11x11 filters applied at stride 4, not padding

Q1: what is the output volume size?

Hint: $W' = (W - F + 2P) / S + 1$

Typical network structures

- AlexNet (8 layers)

Architecture:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

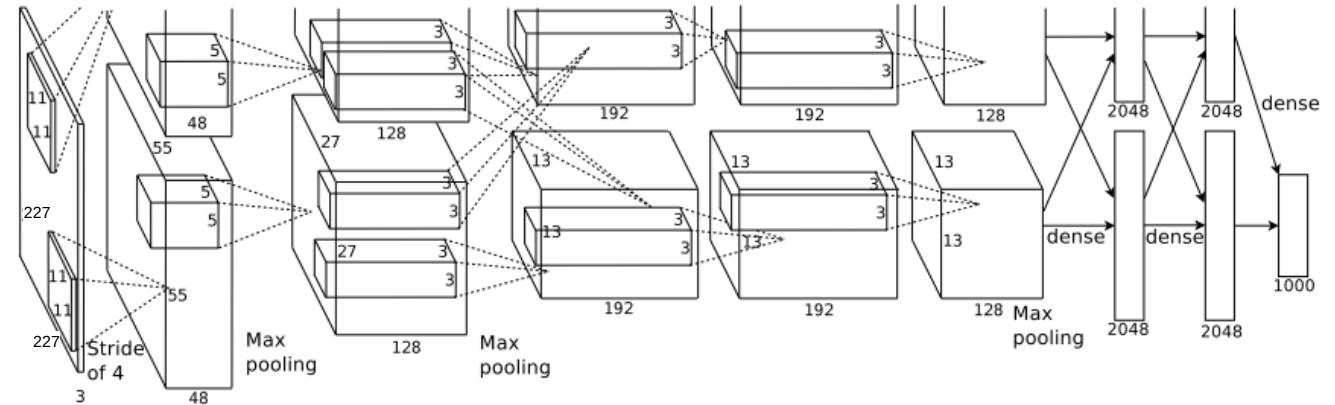
Max POOL3

FC6

FC7

FC8

Input:
227x227x3
images

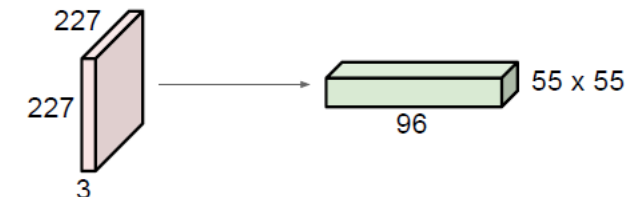


First layer (CONV1): 96 11x11 filters applied at stride 4, not padding

Q1: what is the output volume size?

Hint: $W' = (W - F + 2P) / S + 1$ $(227-11)/4+1 = 55$

Output volume [55x55x96]



Typical network structures

- AlexNet

Architecture:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

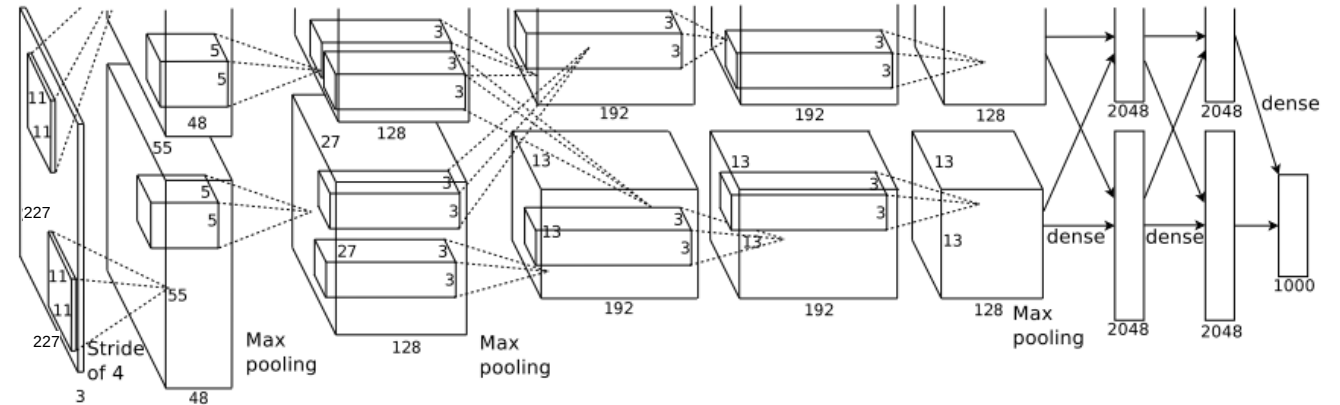
Max POOL3

FC6

FC7

FC8

Input:
227x227x3
images



First layer (CONV1): 96 11x11 filters applied at stride 4, not padding

Q2: what is the total number of parameters in this layer?

Parameters: $11 \times 11 \times 3 \times 96 = 35k$

Typical network structures

- AlexNet

Architecture:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

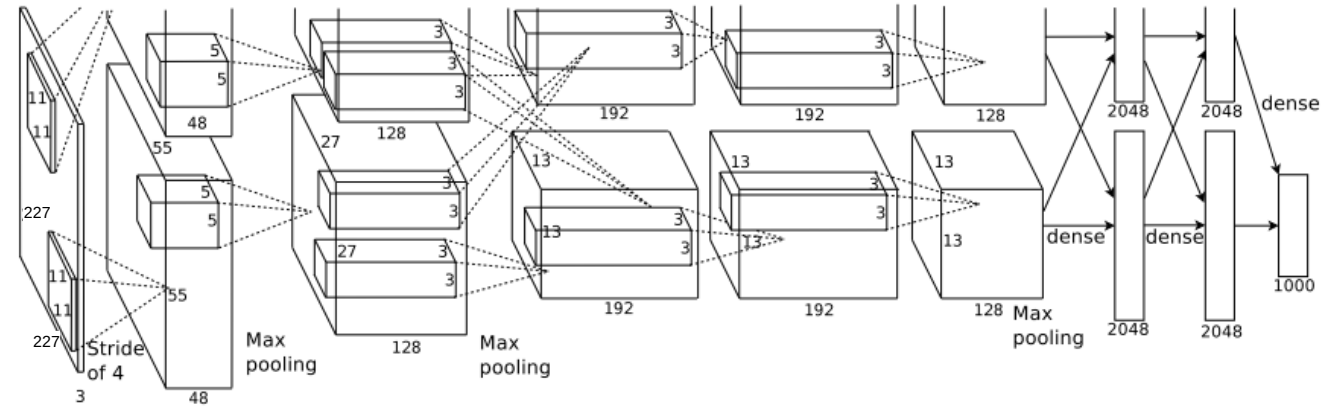
Max POOL3

FC6

FC7

FC8

Input:
227x227x3
images



After CONV1: 55x55x96

Second layer (Pool1): 3x3 filters applied at stride 2

Output volume: 27x27x96

$$(55-3)/2 + 1 = 27$$

Parameters: 0

Typical network structures



- AlexNet

Architecture (simplified):

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

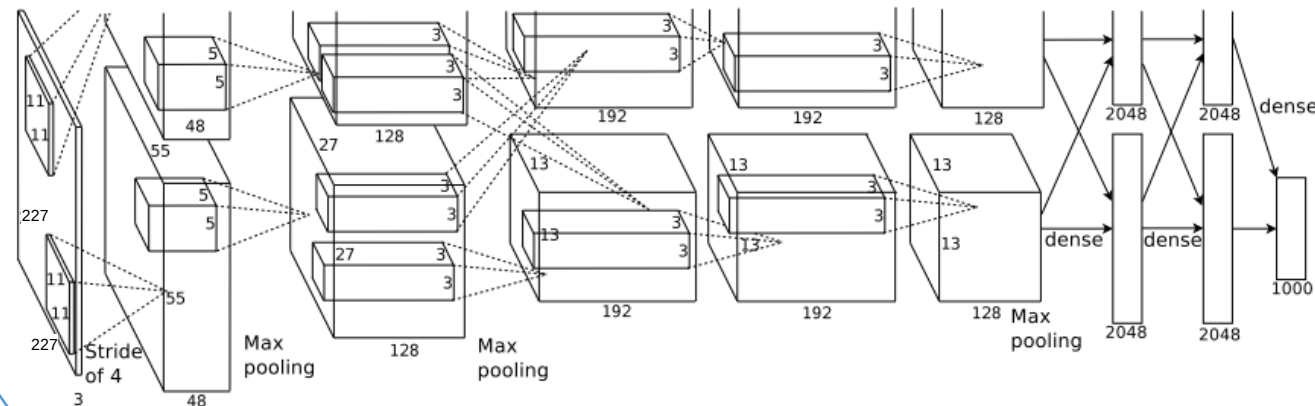
[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAXCONV5 POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)



Conv1 Parameters: $11 \times 11 \times 3 \times 96 = 35k$

FC7 Parameters: $4096 \times 4096 = 16.8m$

参数主要分布在全连接层，卷积层的参数相对较少

Typical network structures

- AlexNet

Architecture (simplified):

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

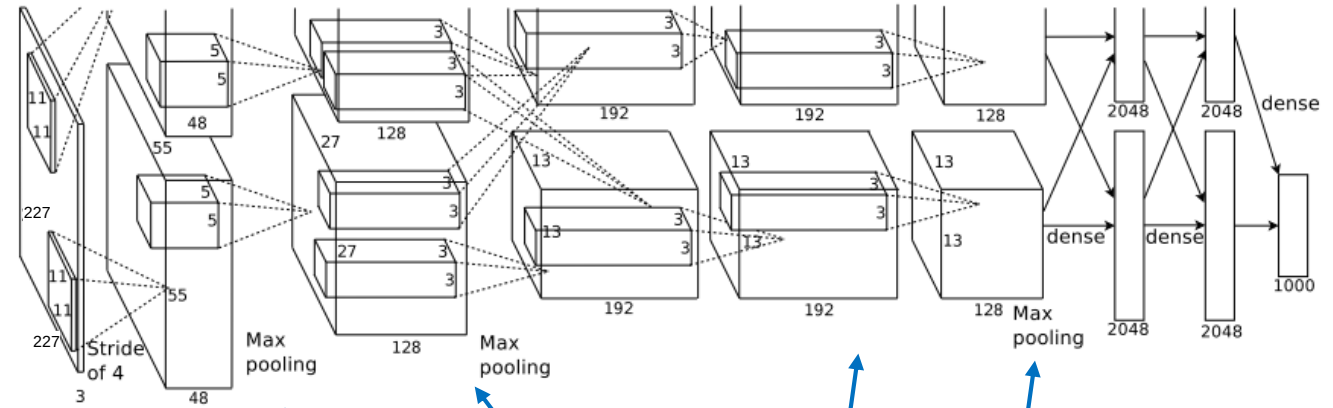
[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX CONV5 POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)



CONV1, CONV2, CONV4, CONV5:
Connections only with feature maps
on same GPU

Typical network structures

- AlexNet

Architecture (simplified):

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

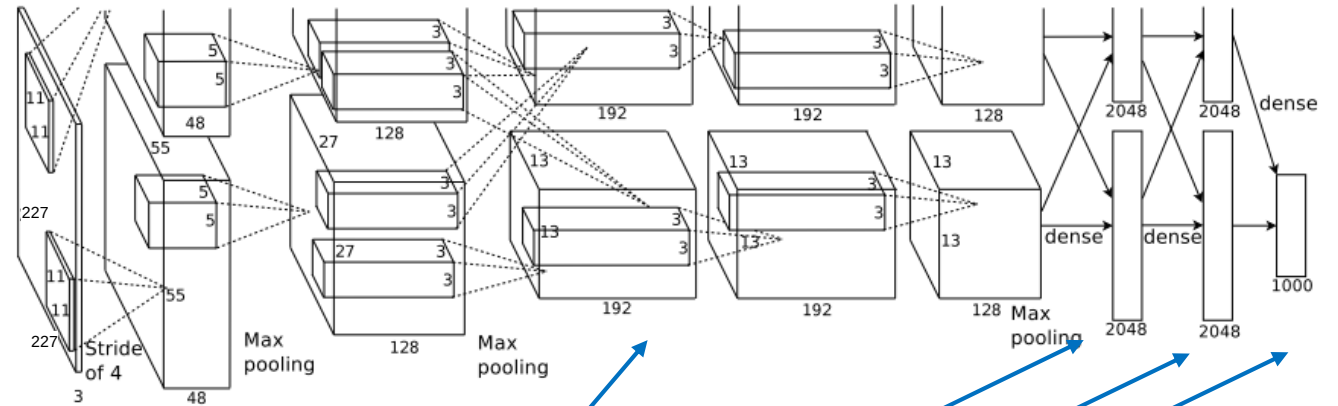
[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAXCONV5 POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)



CONV3, FC6, FC7, FC8:
Connections with all feature maps in
preceding layer, communication
across GPUs

Typical network structures

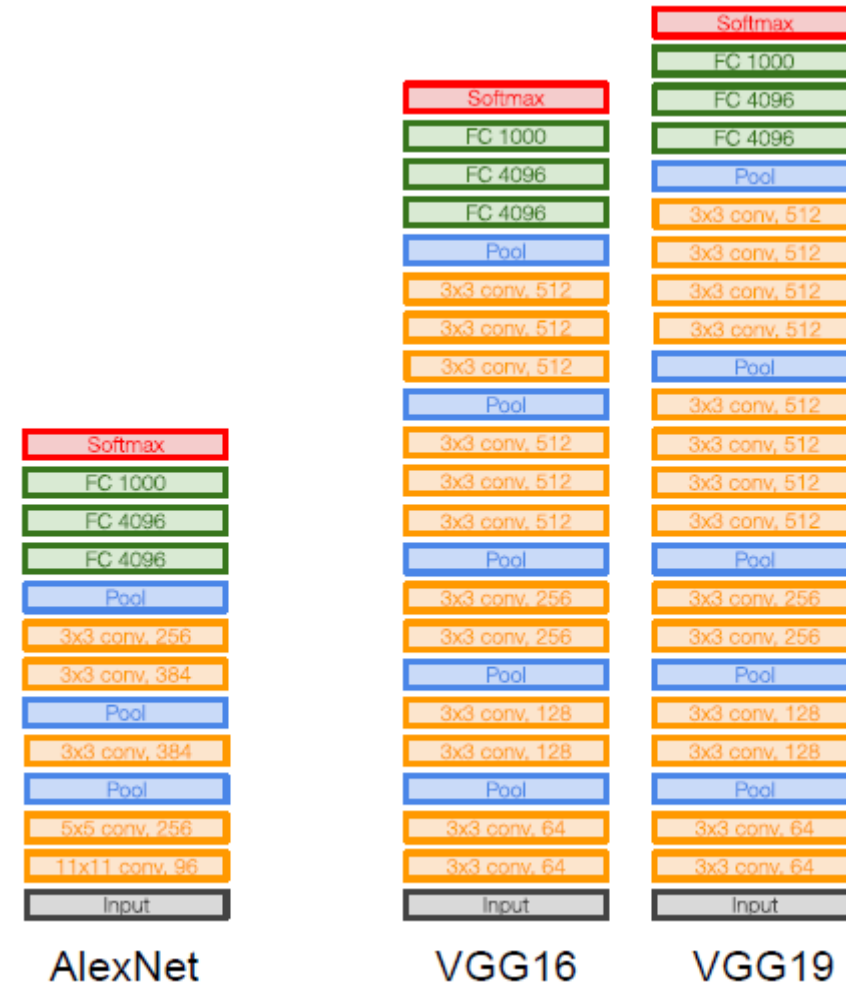


- VGG

- Small filters, deeper networks
- All the convolution use 3x3 kernels
- 5 blocks
 - Each block has 1 – 4 convs
 - Each block followed by max pooling

- Most widely used network

- VGG16
- VGG19



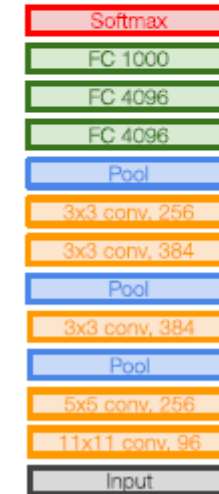
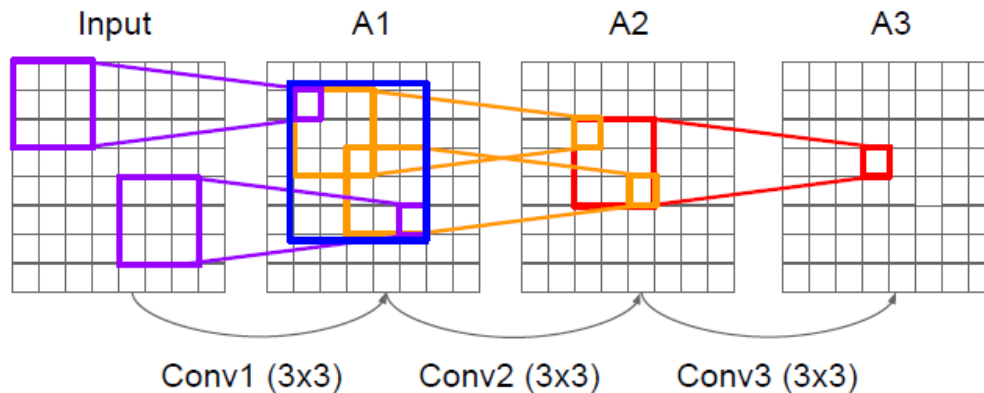
Typical network structures

- VGG

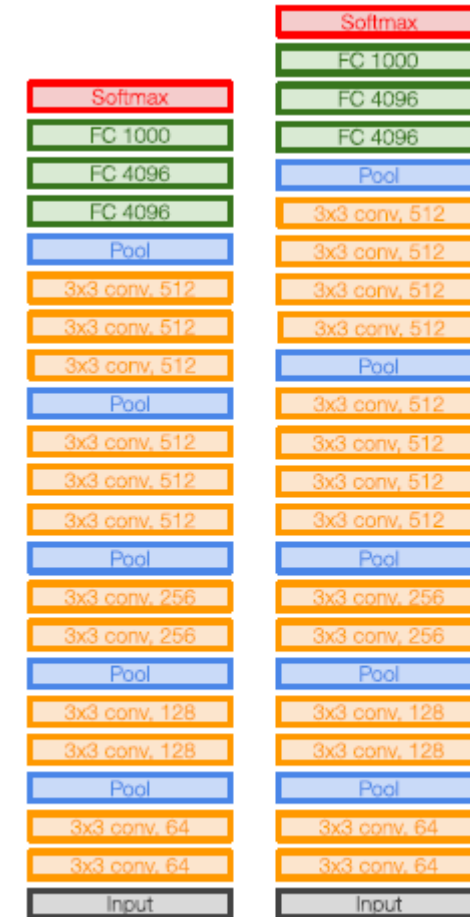
- Small filters, deeper networks
- All the convolution use 3x3 kernels

- Why smaller filters (3x3)?

- Stack of three 3x3 conv (stride 1) layers has same effective receptive field as one 7x7 conv layer



AlexNet

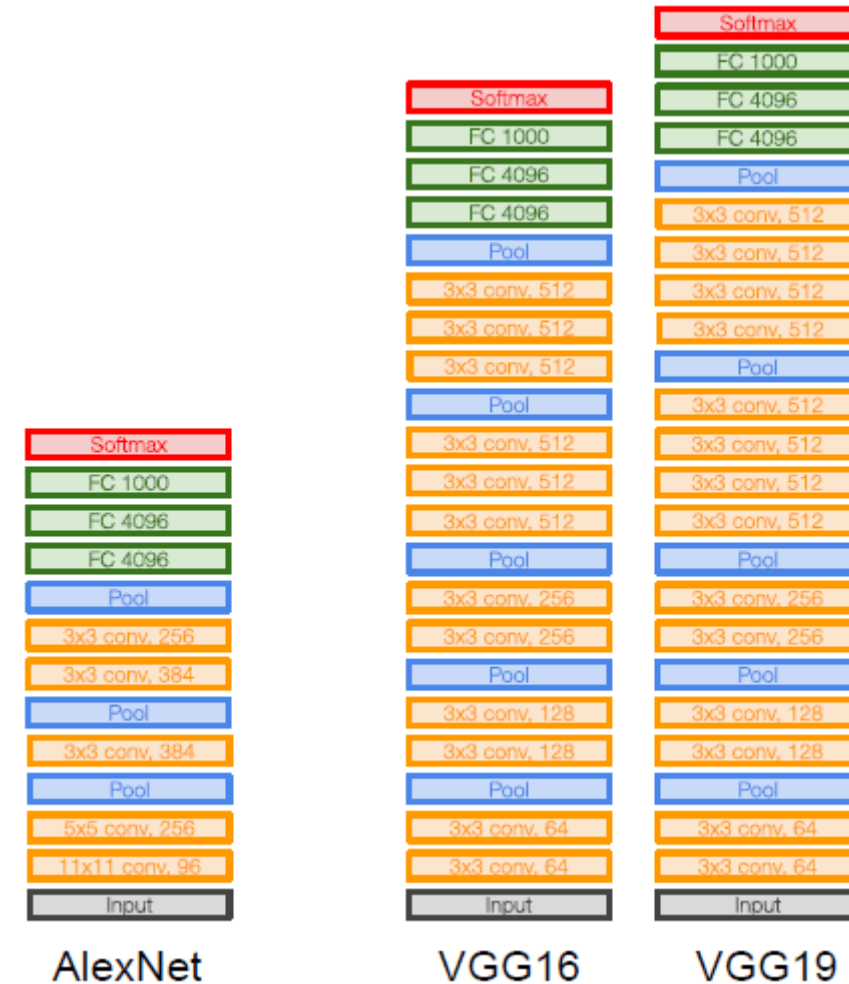


VGG16

VGG19

Typical network structures

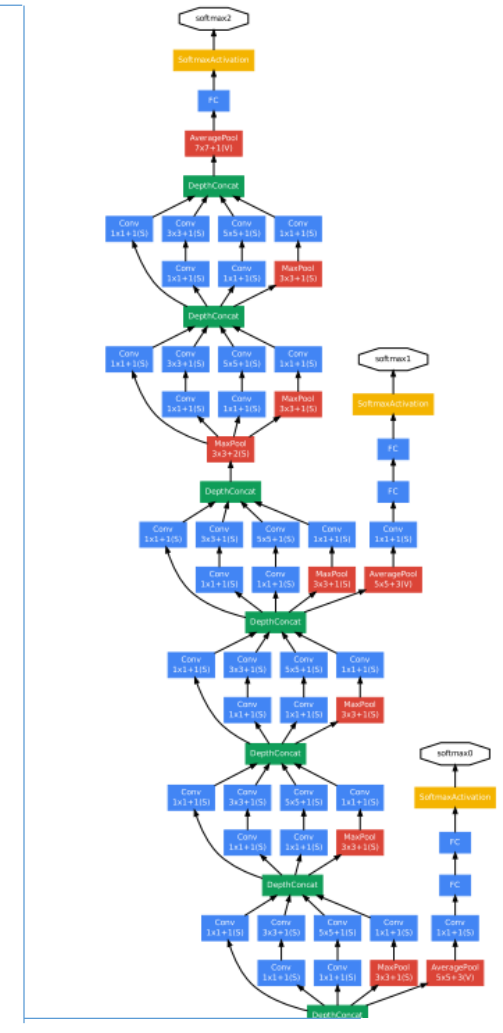
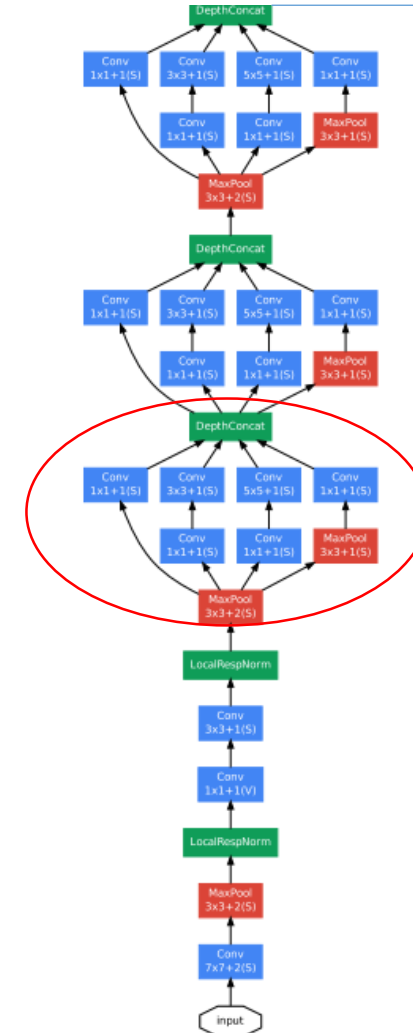
- VGG
 - Small filters, deeper networks
 - All the convolution use 3x3 kernels
- Why smaller filters (3x3)?
 - Stack of three 3x3 conv (stride 1) layers has same effective receptive field as one 7x7 conv layer
- But deeper, more non-linearities
- And fewer parameters: $3 * (3^2 C^2)$ vs. $7^2 C^2$ for C channels per layer



Typical network structures

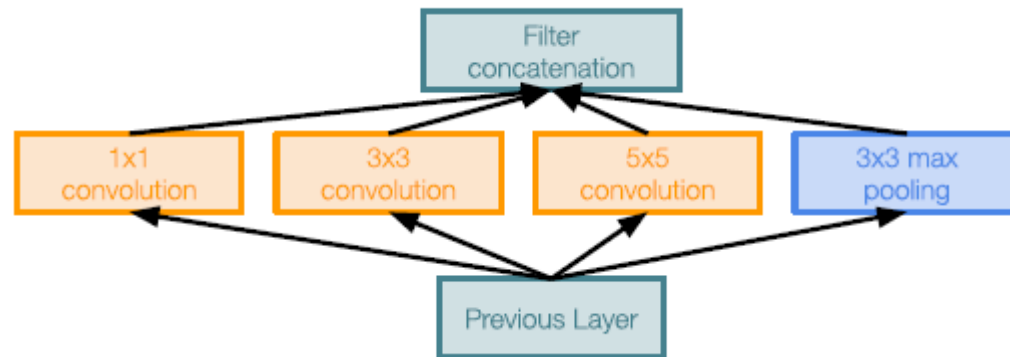
- GoogleNet

- A 22 layers deep network
- Use **Inception** modules to increase width
 - Network within a network
 - A good local network topology and then stack these modules on top of each other



Typical network structures

- GoogleNet



Naive Inception module

并行分支，可提取多尺度特征

Apply parallel filter operations on the input from previous layer:

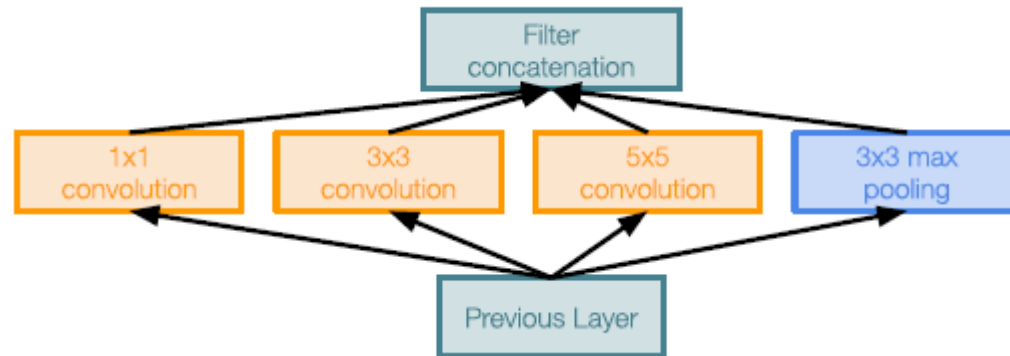
- Multiple receptive field sizes for convolution (1x1, 3x3, 5x5)
- Pooling operation (3x3)
- Concatenate all filter outputs together channel-wise

Problem: High computational complexity

Typical network structures

- GoogleNet

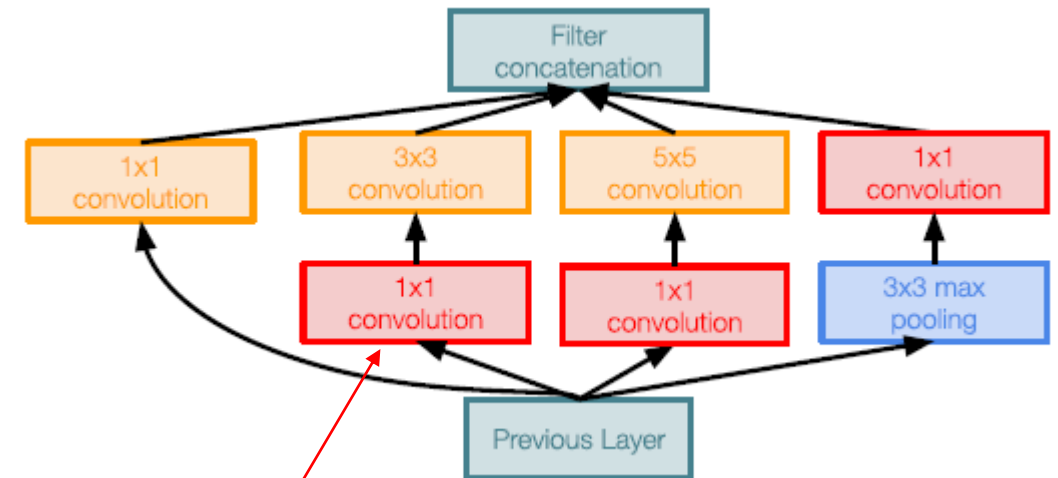
Naive Inception module



Apply parallel filter operations

- Multiple receptive field sizes for convolution (1x1, 3x3, 5x5); Pooling operation (3x3)
- Concatenate all filter outputs together

Inception module with dimension reduction



1x1 conv "bottleneck" layers

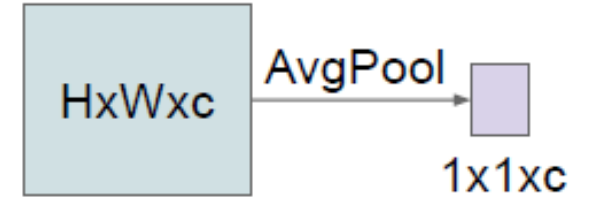
- GoogleNet



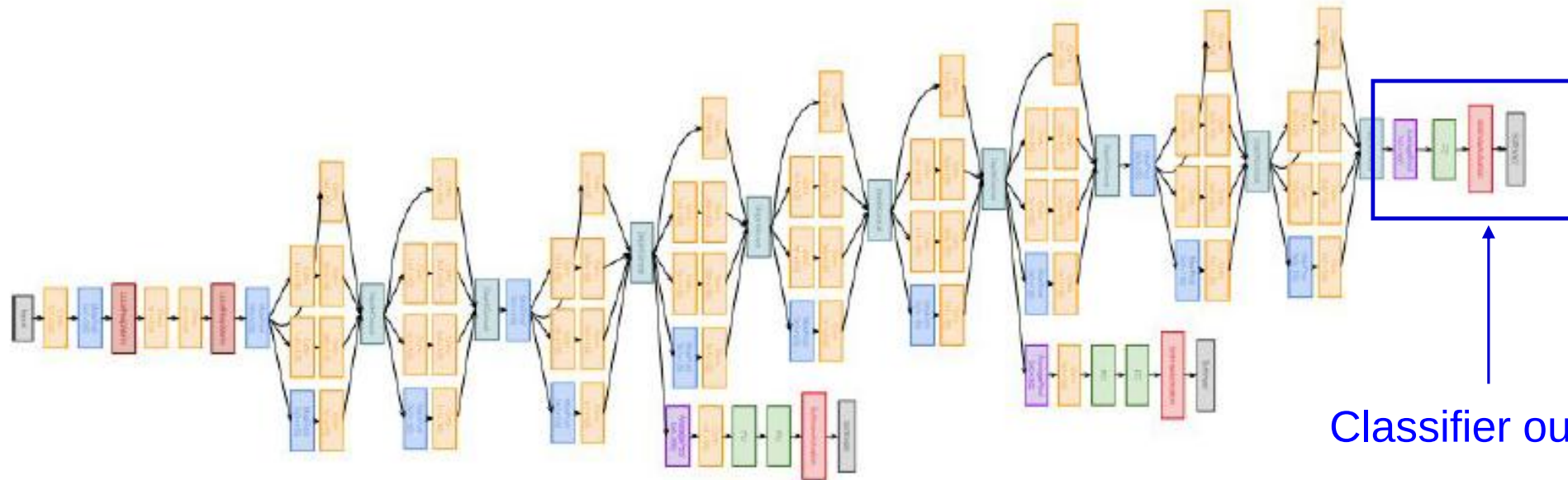
Typical network structures

- GoogleNet

- Global average pooling after the last convolutional layer
- Spatially averages across each feature map
- No longer multiple expensive FC layers!



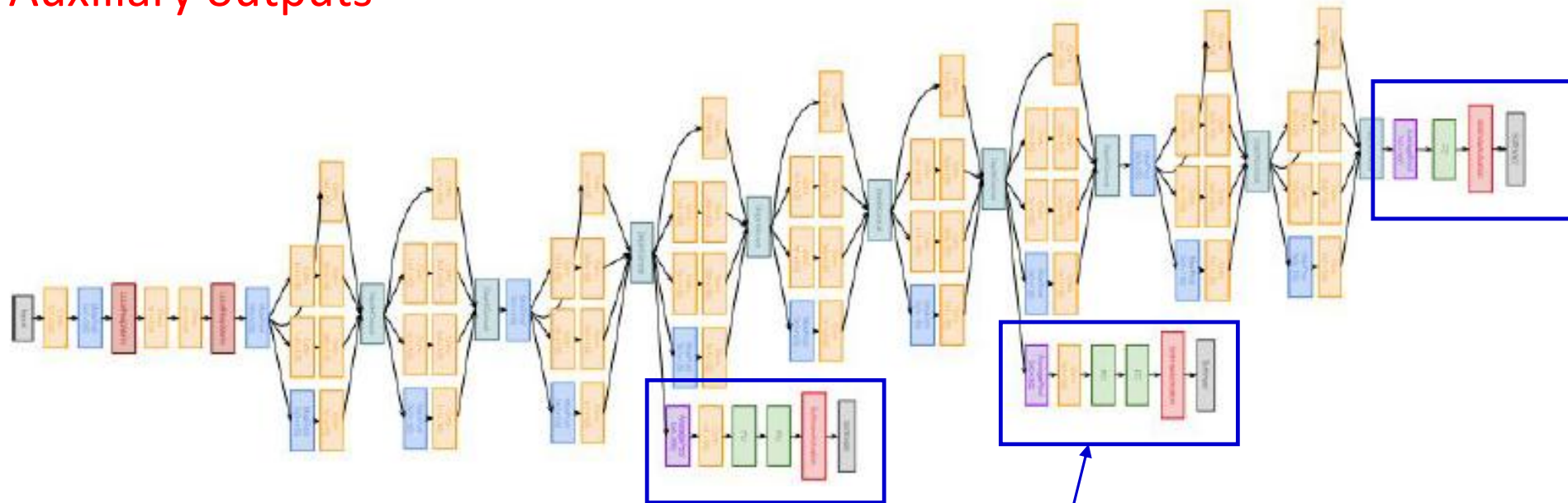
用全局平均池化替代MLP，
减少模型参数



Classifier output

Typical network structures

- GoogleNet
 - Auxiliary outputs



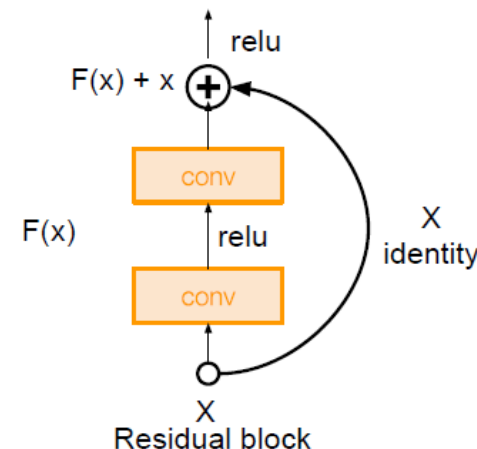
辅助分类器，加快浅层参数更新

Auxiliary classification outputs to inject additional gradient at lower layers

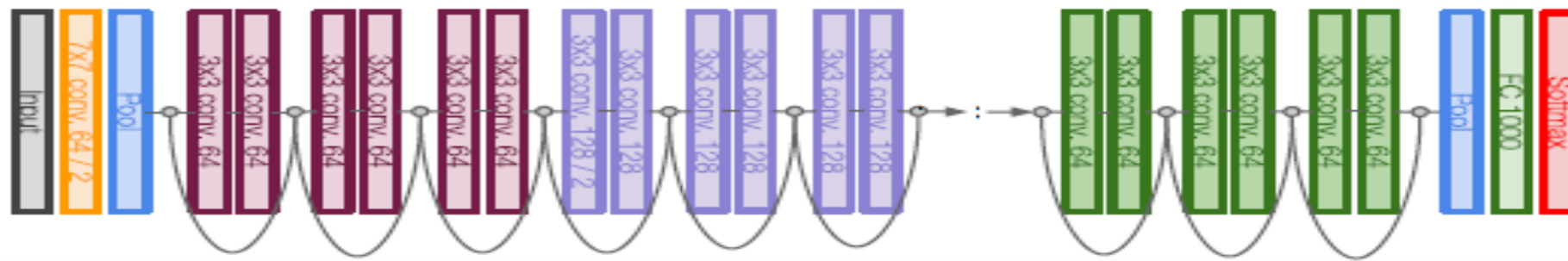
Typical network structures

- ResNet 残差网络

- 通过残差连接可以加快信号反向传播
- 使用了152层的神经网络
- ImageNet 2015年分类冠军

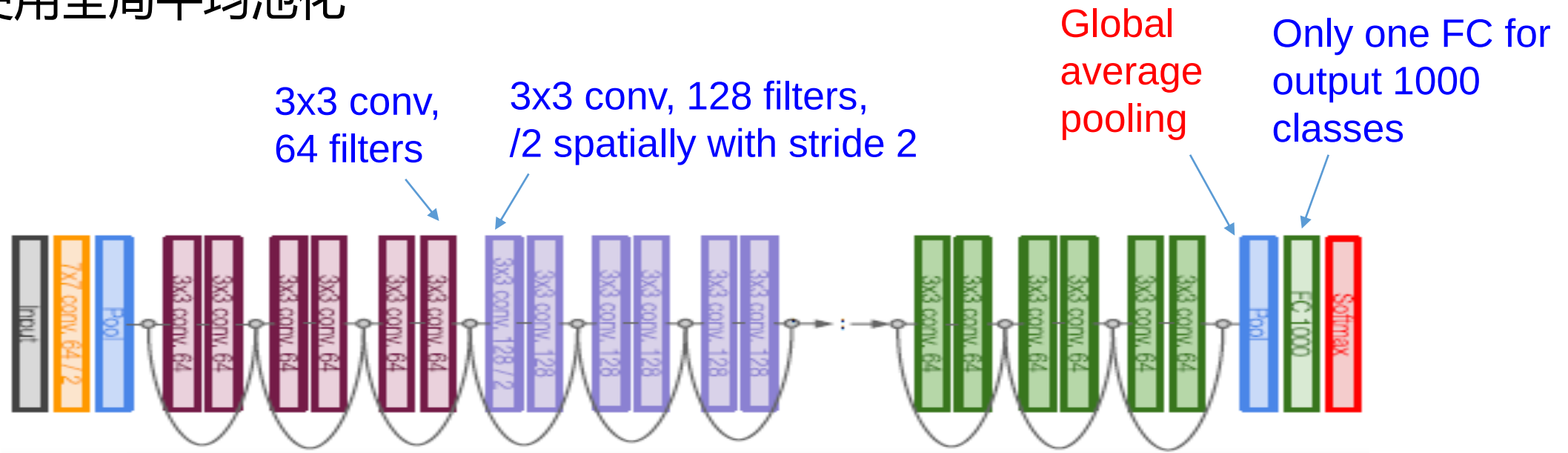


残差连接可加快网络的收敛



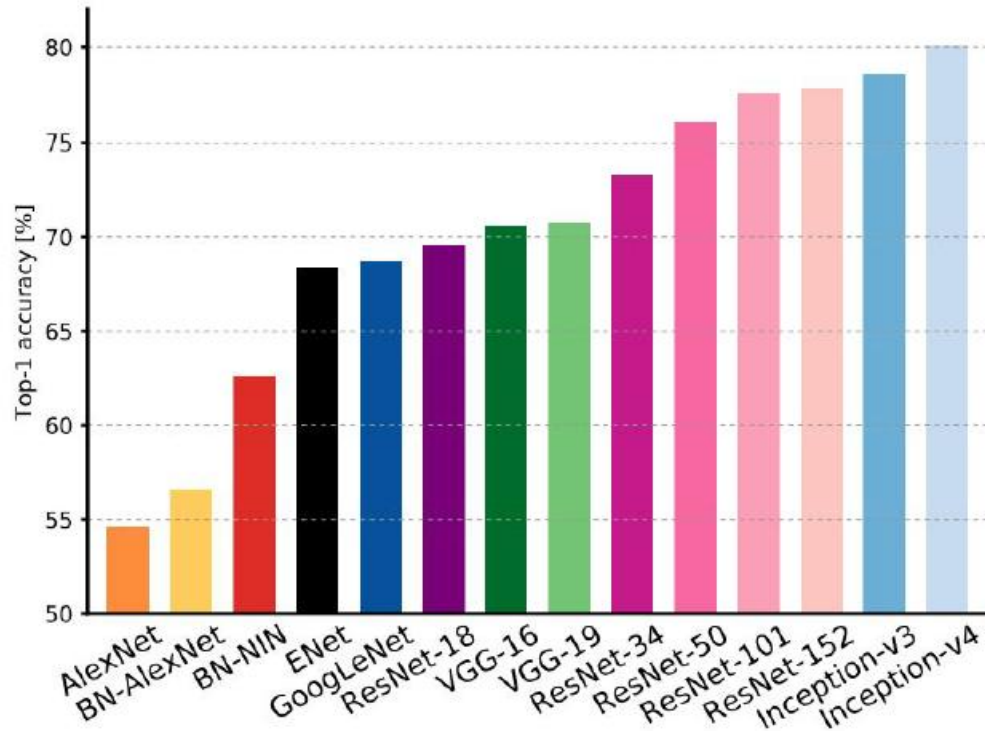
Typical network structures

- ResNet 残差网络
 - 堆叠的残差模块
 - 每个模块有两个卷积层
 - 相邻的两个模块通道数x2, 分辨率/2
 - 使用全局平均池化



Typical network structures

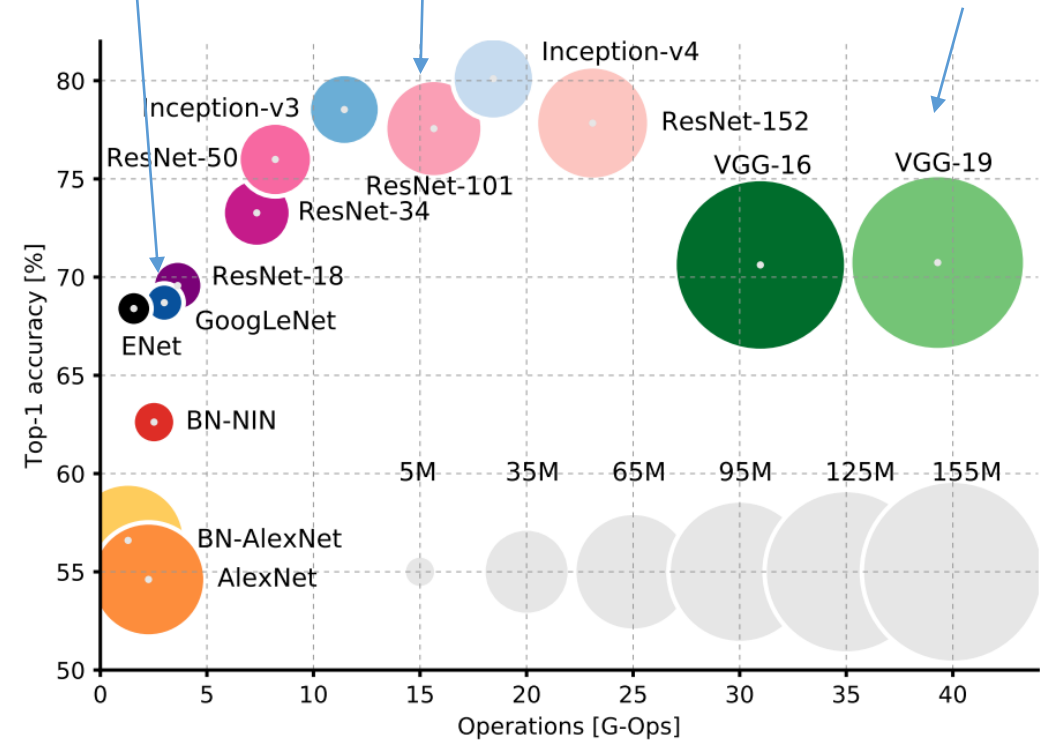
• 不同网络的比较



GoogLeNet:
most efficient

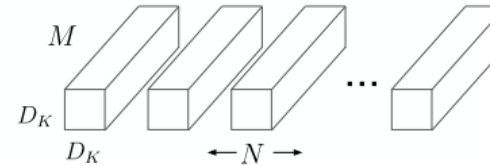
ResNet: Moderate
efficiency, highest
accuracy

VGG: most
parameters and
operations

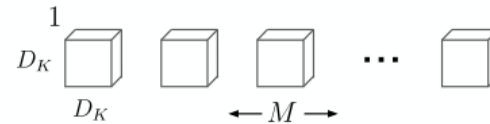


Typical network structures

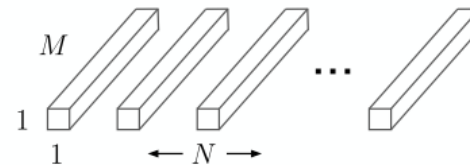
- **MobileNet** 使用深度可分离卷积减少模型参数
 - Use depth-wise separable convolution



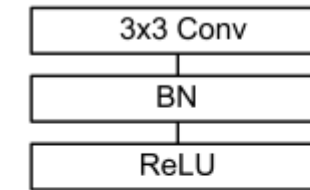
(a) Standard Convolution Filters



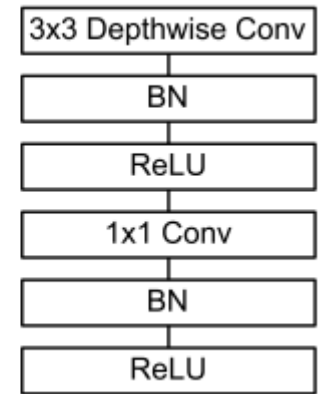
(b) Depthwise Convolutional Filters



(c) 1×1 Convolutional Filters called Pointwise Convolution in the context of Depthwise Separable Convolution



Standard convolution layer



Depthwise and Pointwise layers

Typical network structures

- MobileNet
 - Use depth-wise separable convolution

Table 4. Depthwise Separable vs Full Convolution MobileNet

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
Conv MobileNet	71.7%	4866	29.3
MobileNet	70.6%	569	4.2



Use MobileNet backbone for object detection

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
5×	Conv dw / s1	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 512$
	Conv dw / s2	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 1024$
	Conv dw / s2	$3 \times 3 \times 1024$ dw
	Conv / s1	$1 \times 1 \times 1024 \times 1024$
	Avg Pool / s1	Pool 7×7
	FC / s1	1024×1000
	Softmax / s1	Classifier

Content



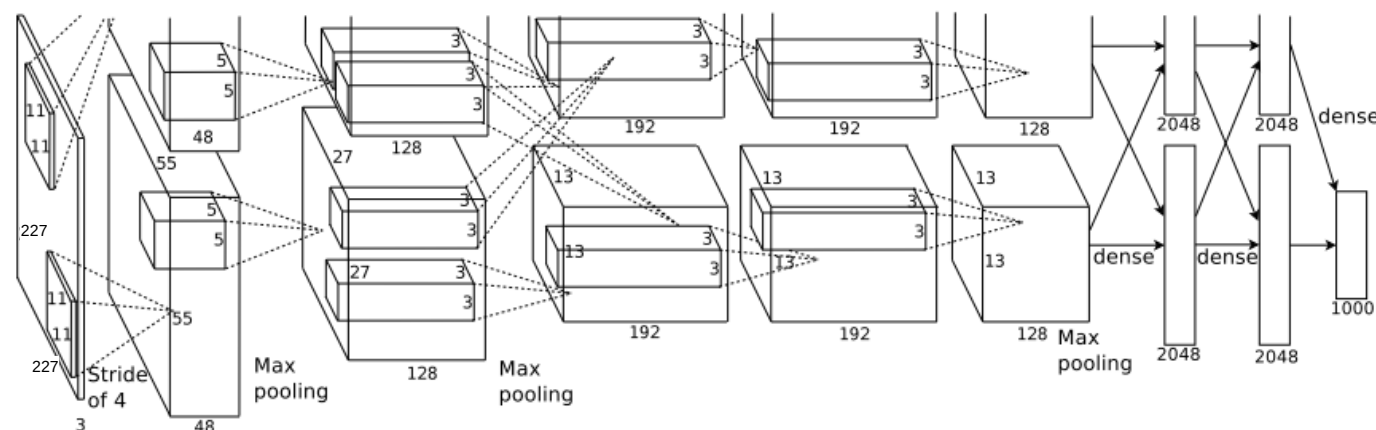
- 典型的图像分类网络
 - AlexNet, VGG
 - GoogleNet, ResNet
 - MobileNet, ShuffleNet
- 对CNN特征的解释

Interpretation of CNN

What's going on inside ConvNets?



Input image

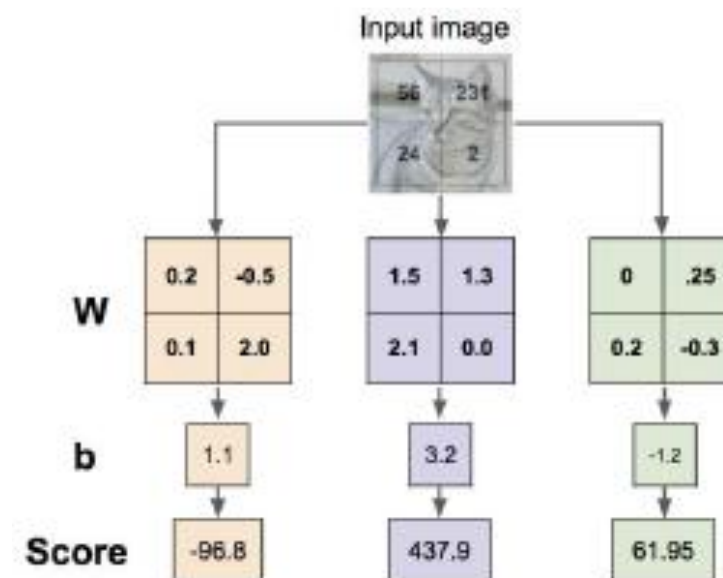


↑ ↑ ↑ ↑ ↑ ↑ ↑

What are the intermediate features looking for?

Interpretation of CNN

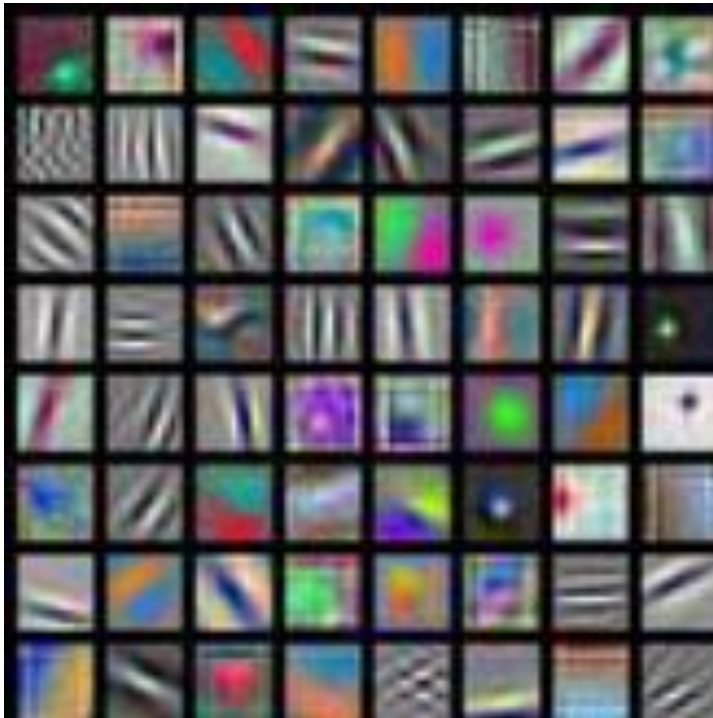
Interpreting a Linear Classifier: Visual Viewpoint



Interpretation of CNN

Visualize Filters

Filters in the first layer



AlexNet: $64 \times 3 \times 11 \times 11$



ResNet-18

$64 \times 3 \times 7 \times 7$



ResNet-101

$64 \times 3 \times 7 \times 7$



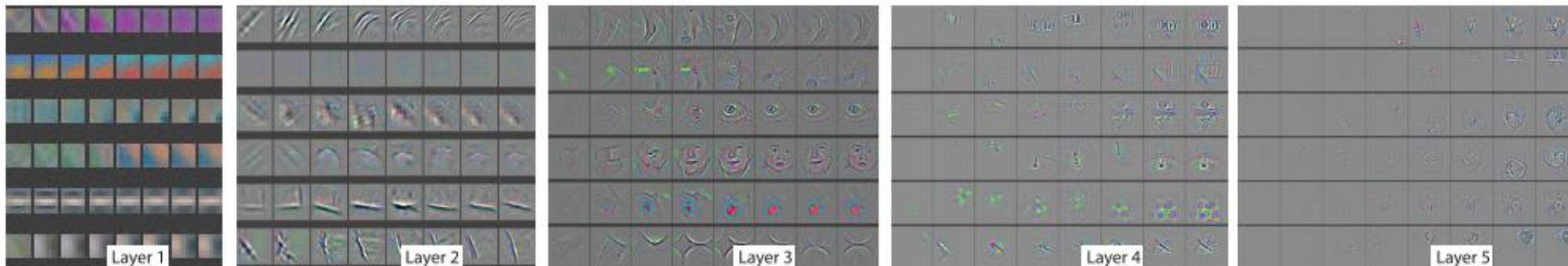
DenseNet-121

$64 \times 3 \times 7 \times 7$

Interpretation of CNN

Visualize Filters

Filters in the intermediate layers



Shallow layers learn some **general low-level patterns**, such as edge filters

Deep layers learn more **specific high-level patterns** associated with objects

Interpretation of CNN

Last Layer: Nearest Neighbors

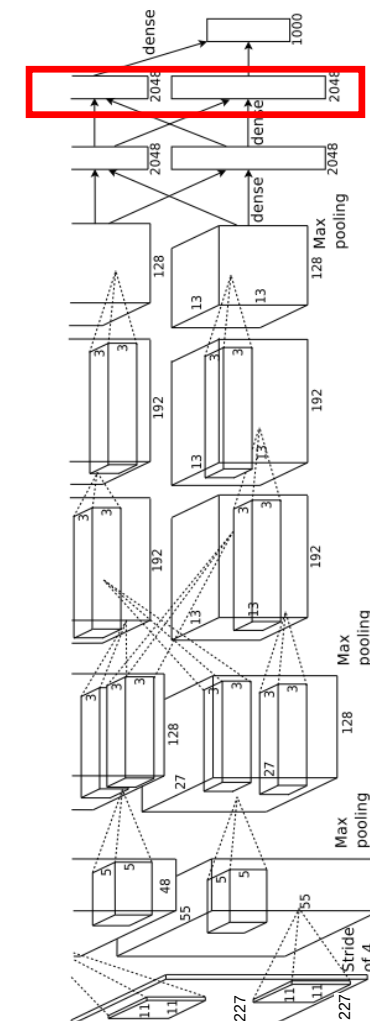
Nearest neighbors in **pixel space**



Test image



L2 nearest neighbors in **feature space**

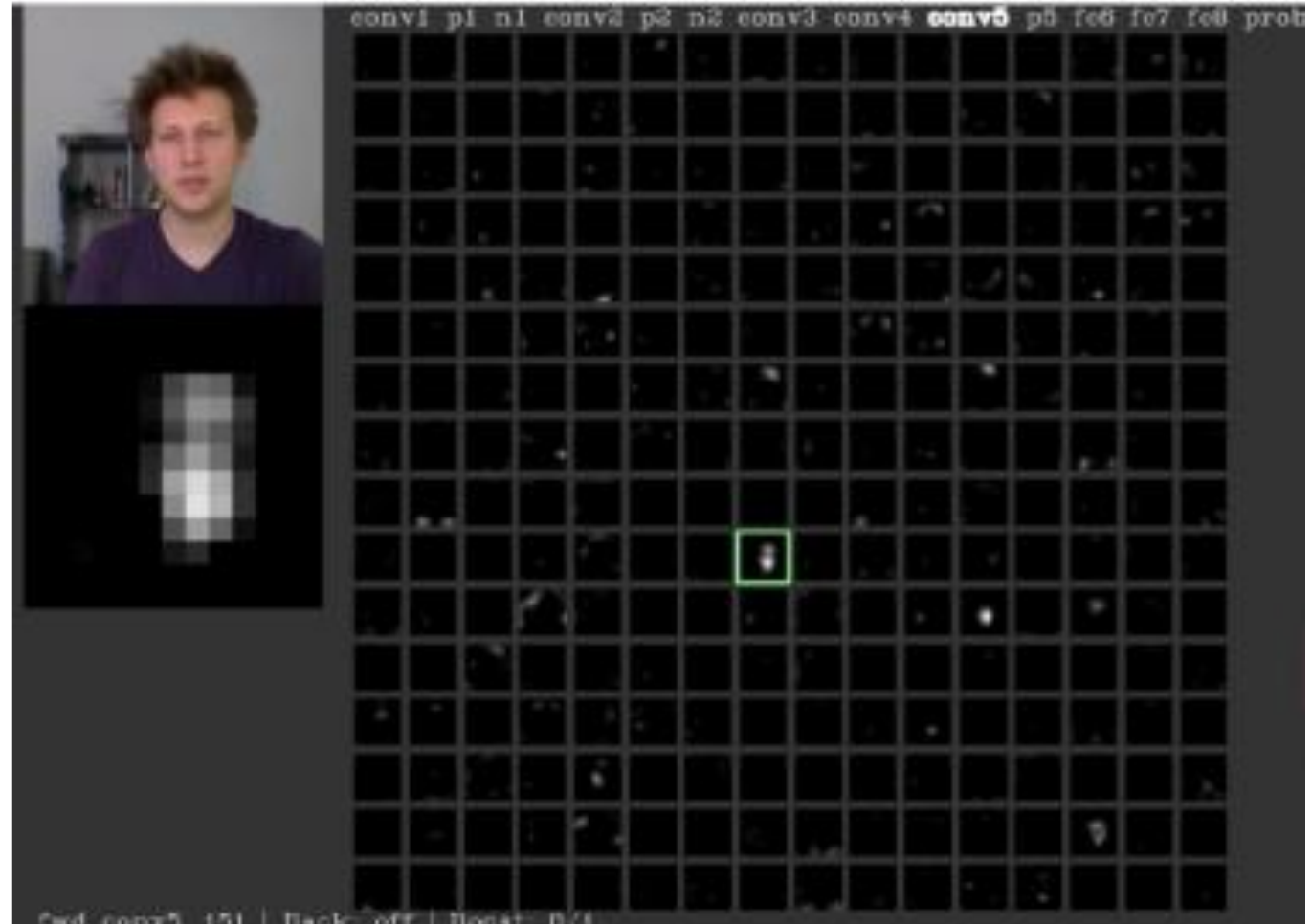


Interpretation of CNN



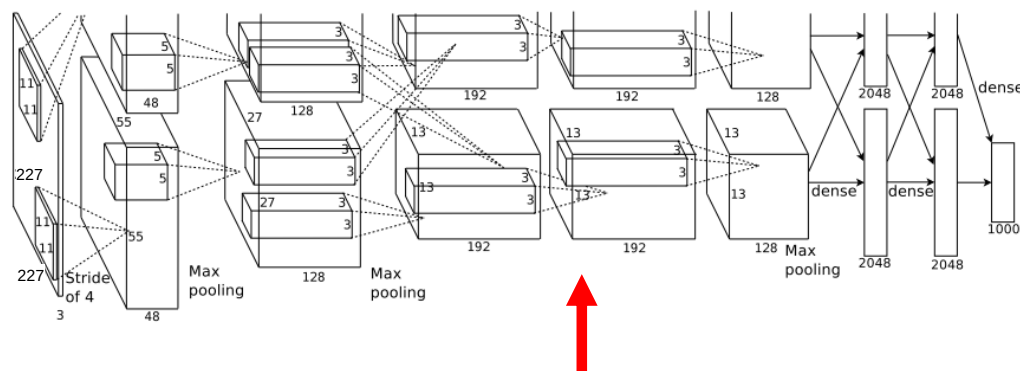
Visualizing Activations

conv5 feature map is 128x13x13;
visualize as 128 13x13 grayscale
images



Interpretation of CNN

Maximally Activating Patches



Pick a layer and a channel; e.g. conv5 is 128 x 13 x 13, pick channel 17/128

Run many images through the network, record values of chosen channel

Visualize image patches that correspond to maximal activations

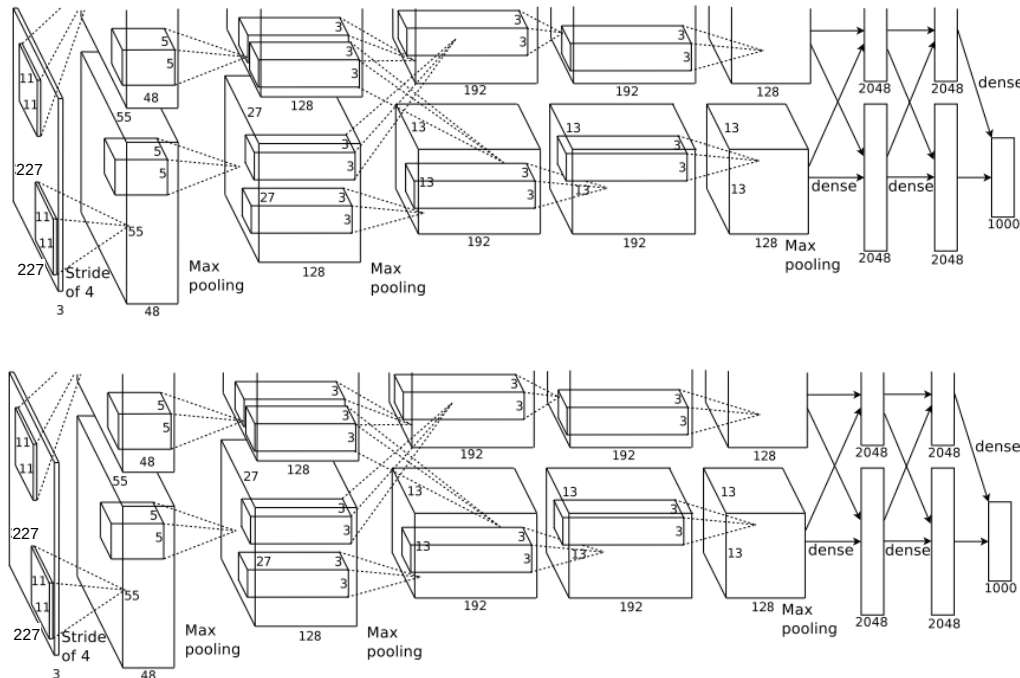


Interpretation of CNN

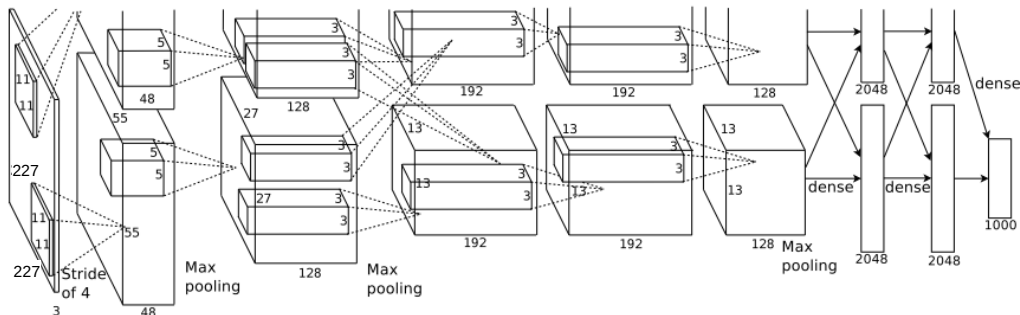
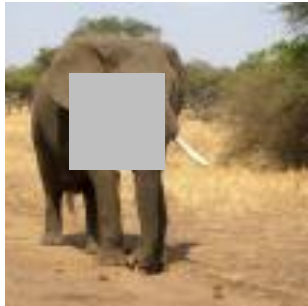
Which pixels matter: Saliency via Occlusion

Mask part of the image before feeding to CNN

Check how much predicted probabilities change



$P(\text{elephant}) = 0.95$



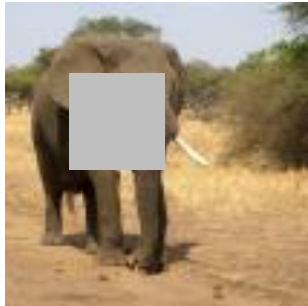
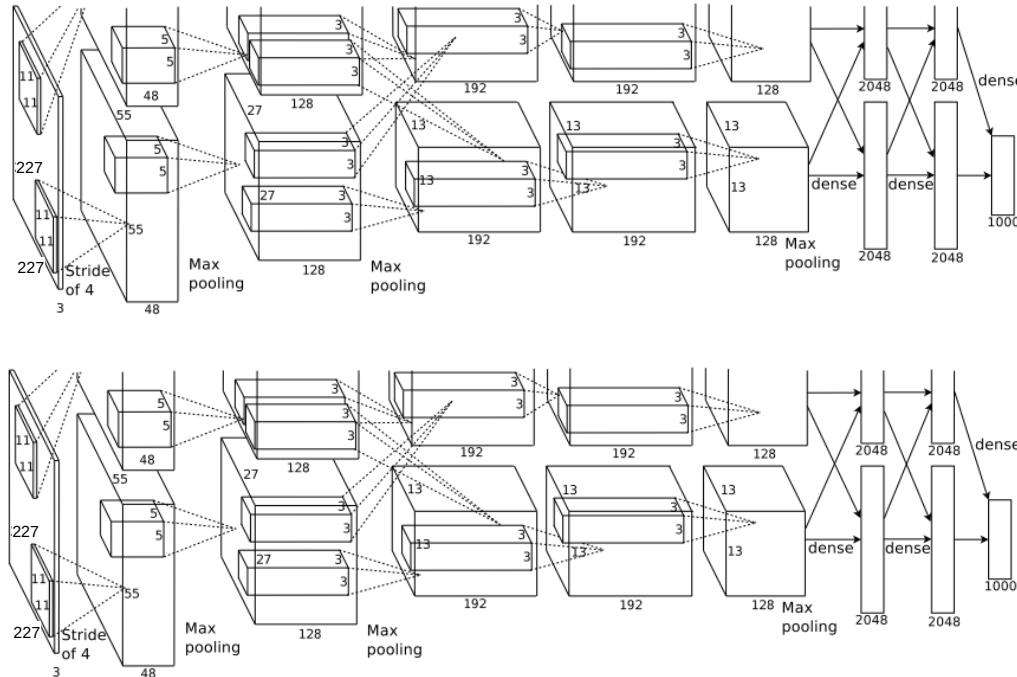
$P(\text{elephant}) = 0.75$

Interpretation of CNN

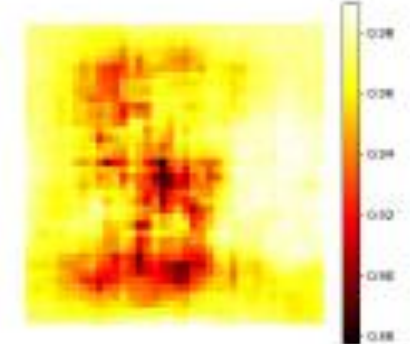
Which pixels matter: Saliency via Occlusion

Mask part of the image before feeding to CNN

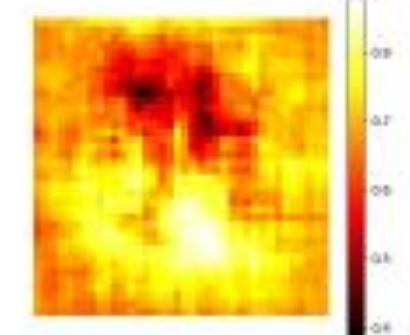
Check how much predicted probabilities change



schooner



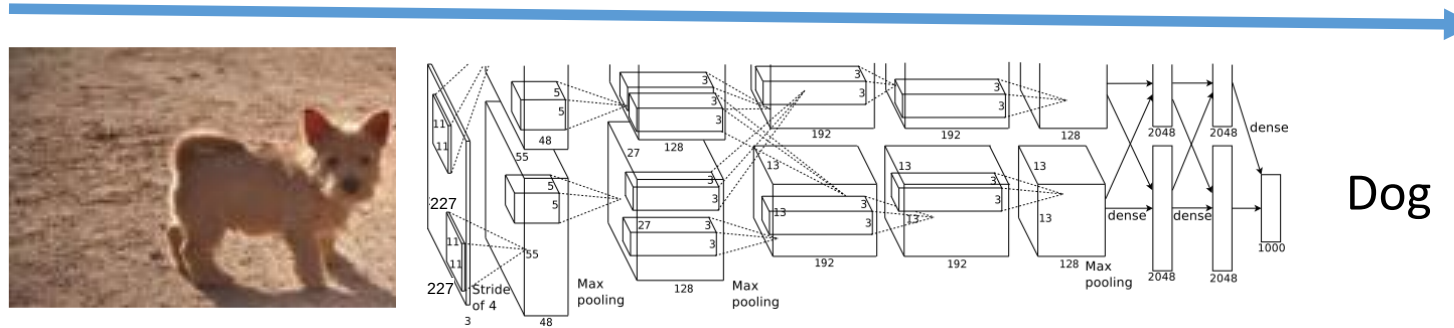
African elephant



Interpretation of CNN

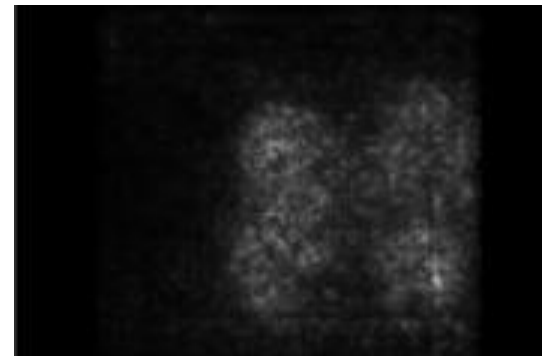
Which pixels matter: Saliency via Backprop

Forward pass: Compute probabilities



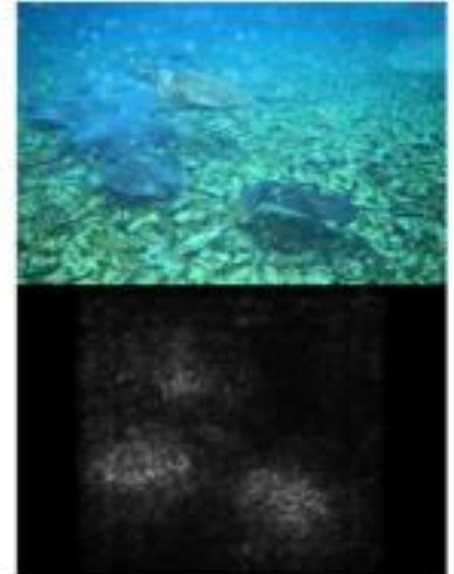
Backward pass

Compute gradient of (unnormalized) class score with respect to image pixels, take absolute value and max over RGB channels



Interpretation of CNN

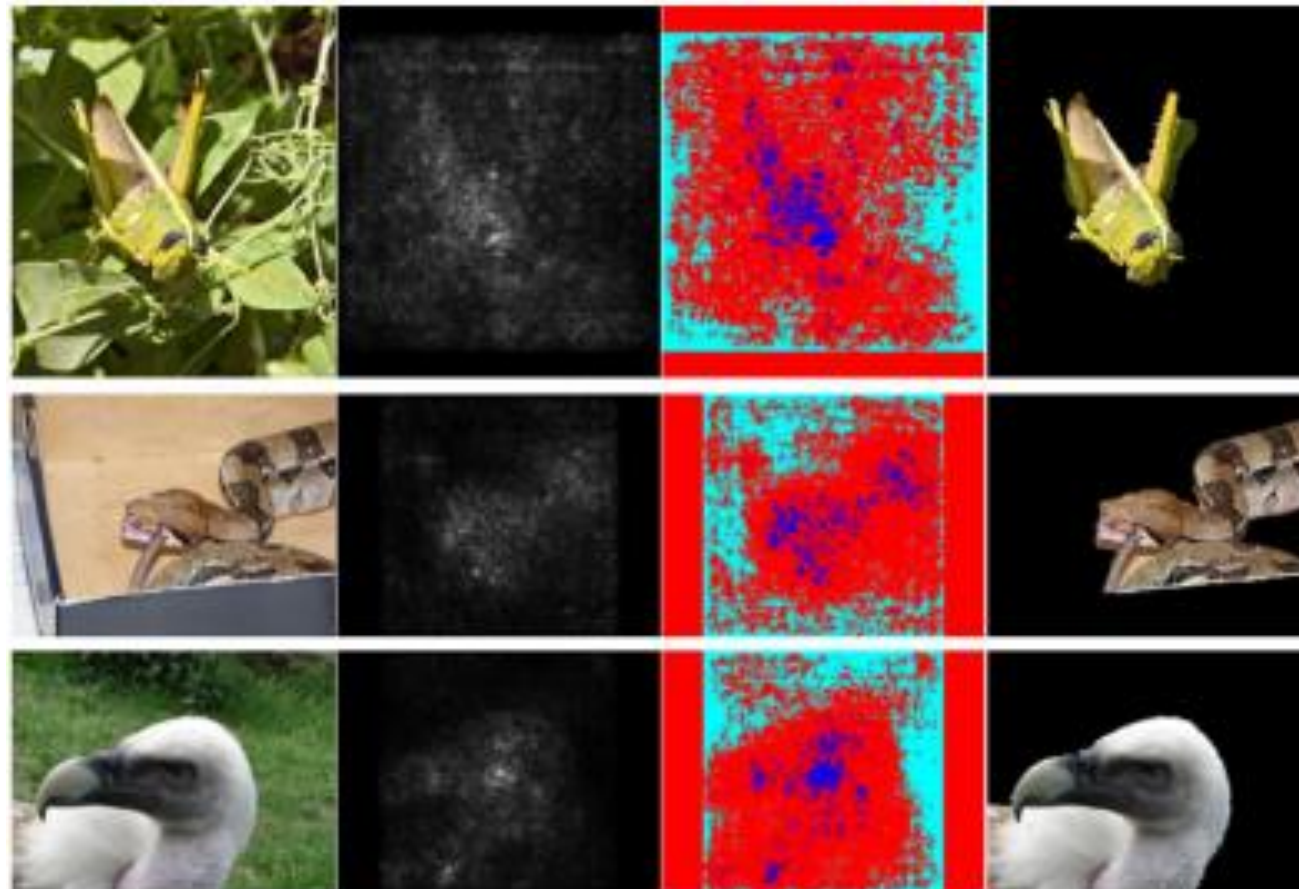
Saliency via Backprop



Interpretation of CNN

Saliency Maps: Segmentation without supervision

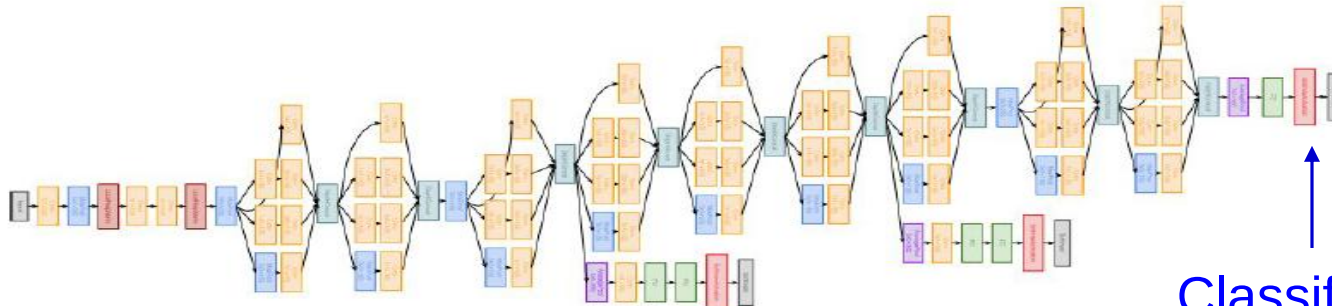
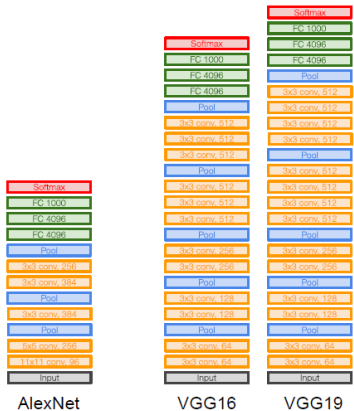
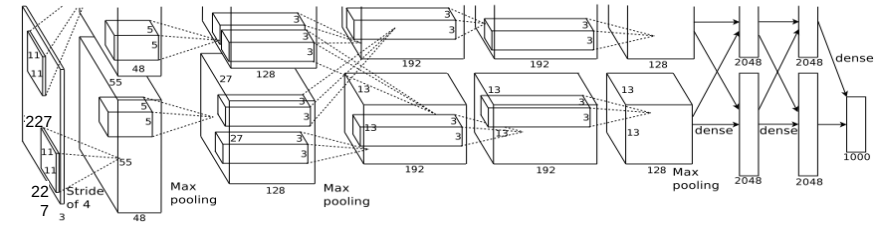
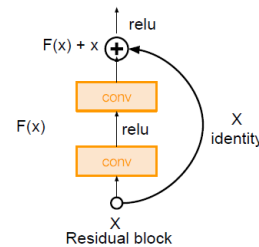
Use GrabCut on
saliency map



小结

- 1, Basics for convolutional neural networks
 - Softmax and Cross entropy
- 2, Typical Network structures
 - AlexNet, VGG
 - GoogleNet, ResNet
 - MobileNet, ShuffleNet
- 3, CNN Understanding

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$
$$L_i = - \sum_{j=1}^K y_j \log p_j$$



Classifier
output

